

# NLP 2026 - Introduction to NLP

Lecture 1 | Foundations of NLP and Large Language Models

Prof. Dr. Paul Pronobis  
Free University of Bozen-Bolzano



# About Your Lecturer

# Getting to Know Each Other

# Professional Career (1/2)

## Prof. Dr. Paul Pronobis

| Period         | Position                                                                    |
|----------------|-----------------------------------------------------------------------------|
| 2024 - present | <b>Full Professor</b> , Free University of Bozen-Bolzano 🇮🇹                 |
| 2021 - 2024    | Full Professor of Accounting, EBS University, Wiesbaden 🇩🇪                  |
| 2019 - 2021    | Associate Professor, ESCP Business School, Paris 🇫🇷                         |
| 2015 - 2019    | Visiting Professor at Indiana University, UT Austin & Rollins College<br>🇺🇸 |
| 2011 - 2017    | Assistant Professor, Freie Universität Berlin 🇩🇪                            |

**Education:** Habilitation, FU Berlin (2019) · Dr. rer. oec., HHL Leipzig (2011) · Diploma, HS Pforzheim (2006) · MBA Exchange, Indiana University (2005)

**Research & interests:** Financial Reporting & Audit Quality · XBRL & Digital Reporting · Earnings Quality & IFRS Adoption · AI, NLP, and agentic systems · **Languages:** 🇩🇪 🇵🇱 🇬🇧 🇮🇹 🇫🇷

# Professional Career (2/2)

## Industry Experience

| Period      | Role                                                       | Key Focus                                                                       |
|-------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| 2010 - 2011 | <b>Senior Manager Accounting</b> , HARMAN Becker (Samsung) | Worldwide consolidated accounting, US-GAAP & HGB, international team leadership |
| 2006 - 2010 | <b>Senior Auditor</b> , Ernst & Young, Stuttgart           | Group audits (US-GAAP, IFRS), software/energy/automotive sectors, team lead     |

 paul.pronobis@unibz.it · Office: E2.01



# Personal Life



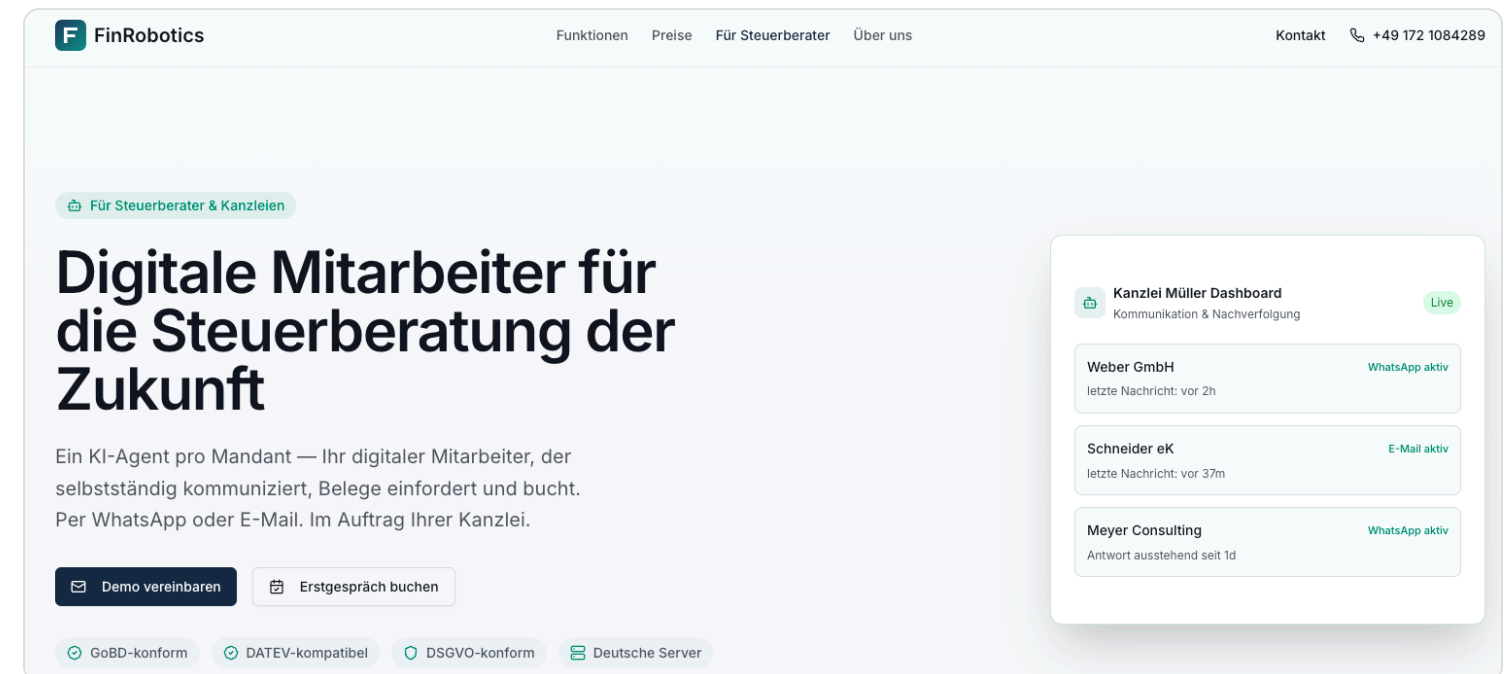
Family of four (+ a Cockapoo 🐶) - based between Karlsruhe 🇩🇪 and South Tyrol 🇮🇹



# Besides Teaching: FinRobotics

## AI Startup at the Intersection of Accounting and AI

- By day: Professor of Accounting 
- On the side: **FinRobotics** – Founder
- AI agents for tax advisory firms
- Automated client communication via WhatsApp & Email
- Accounting × Technology × AI workflows







# Lecture 1 Roadmap

# Introduction to NLP

# Today in one sentence

## The six parts

---

1. Course overview
2. What is NLP?
3. NLP core concepts
4. Hugging Face and pipelines
5. Hands-on session
6. Wrap-up

## Teaching logic

---

- Concept first
- Example second
- Tool third
- Practice fourth
- Reflection at the end

# Today in one sentence, takeaway

We build a practical mental model of what NLP is, how modern NLP workflows operate, and how to work with language models critically and hands-on rather than treating them as magic.





# Part 1

# Course overview

# Welcome to the module

## NLP and Web Analytics inside Big Data Methods

---

- Module **M2** of **27512 Big Data Methods for Economics and Business**
- Master in Data Analytics for Economics and Management, year 2
- Focus on **practical NLP, modern LLM workflows, and business use cases**
- Live sessions for interaction, coding, and discussion
- Video recordings for structured deep dives after class

# Welcome to the module, core principle



## Core principle

This module is designed to make NLP **usable, critical, and concrete**.

# Why this course matters now

## Language became infrastructure

---

- Search is language
- Chat interfaces are language
- Customer support is language
- Knowledge work is language
- AI products increasingly depend on language models

## For business students, that means

---

- understanding opportunities
- spotting weak claims and hype
- prototyping useful workflows
- communicating with technical teams
- judging cost, quality, and risk

# About this module

## What you will learn

---

- how NLP fits into AI and ML
- how basic NLP pipelines work
- how to use pre-trained models responsibly
- how to connect language tasks to business problems

## How we will work

---

- slides for structure
- notebooks for doing
- discussion for judgment
- assignments for transfer



# Module roadmap

| Module                                      | Main question                                  |
|---------------------------------------------|------------------------------------------------|
| 1. Introduction to NLP                      | What is NLP and how do we start?               |
| 2. Text Classification & Sentiment Analysis | How can we assign labels to text?              |
| 3. Neural Networks & Language Modeling      | Why do transformers work so well?              |
| 4. Information Retrieval & RAG              | How do we retrieve relevant knowledge?         |
| 5. Web Scraping                             | How do we collect text data?                   |
| 6. Prompt Engineering                       | How do we guide model behavior better?         |
| 7. Fine-Tuning                              | When is adaptation better than prompting?      |
| 8. LLM Applications & AI Agents             | How do modern AI systems go beyond one prompt? |

# Live schedule

| Session | Date       | Focus                                         |
|---------|------------|-----------------------------------------------|
| L1      | 17.04.2026 | Course intro + Introduction to NLP + hands-on |
| L2      | 29.04.2026 | Text classification and sentiment analysis    |
| L3      | 12.05.2026 | Information retrieval and RAG                 |
| L4a     | 13.05.2026 | Web scraping for knowledge construction       |
| L4b     | 13.05.2026 | Prompt engineering workshop                   |
| L5      | 04.06.2026 | Fine-tuning                                   |
| L6      | 05.06.2026 | LLM applications, AI agents, exam review      |



# Assessment

| Component     | Weight | Notes                                                           |
|---------------|--------|-----------------------------------------------------------------|
| Assignments   | 40%    | Four computer-based assignments, mostly notebook-oriented       |
| Final exam    | 60%    | Written exam, conceptual understanding plus practical reasoning |
| Best strategy | -      | <b>Practice actively every week, do not cram at the end</b>     |

# Assignment logic

## Assignments

---

- **A1** NLP basics and classification
- **A2** IR and RAG
- **A3** scraping and prompt engineering
- **A4** fine-tuning or LLM application

## What I care about

---

- clean reasoning
- readable code
- sensible choices
- critical interpretation
- not just getting any output

# Tools we will use

- Python
- Jupyter notebooks
- Hugging Face Transformers
- `pandas`, `matplotlib`, `scikit-learn`
- selected APIs for modern workflows
- EDUNEXT for materials and recordings

# What I expect from you

- be willing to experiment
- ask when the logic is unclear
- test model outputs critically
- connect technical methods to business value
- accept that first attempts are often messy

# What I expect from you, mindset



## Mindset

Treat notebooks as laboratories, not as polished museum objects.

# Opening prompt

## Connect your experience to the course

---

1. Where do you already use NLP in daily life?
2. What do you hope to understand better by the end of this course?

Then we collect a few examples in class.



# What is Artificial Intelligence?

**AI = computer systems that solve tasks which normally require human intelligence.**

## Three categories

| Category                            | Status         | Description                                                             |
|-------------------------------------|----------------|-------------------------------------------------------------------------|
| <b>Weak AI</b> (Narrow AI, ANI)     | ✅ In use today | Specialized for one task (voice assistants, image recognition, ChatGPT) |
| <b>General AI</b> (General AI, AGI) | 💡 Hypothetical | Can do everything a human can do. Not achieved yet.                     |
| <b>Superintelligence</b> (ASI)      | ⚡ Speculative  | Surpasses humans in all areas                                           |

### ⚠ Important distinction

AI ≠ programming ≠ automation. AI **learns from data**, traditional software follows fixed rules.

# From AI to LLMs

Everything we work with today (ChatGPT, Claude, Gemini) are **LLMs**:

- **LLM** = Large Language Model
- **Large**: Billions to trillions of parameters
- **Language**: Trained on human language (web, books, code)
- **Model**: A mathematical representation of probabilities

## The breakthrough: 2017

Google introduces the **Transformer** architecture. It allows AI to “see” the context of an entire sentence at once, instead of reading word by word.



# 2017: The paper that changed everything

**Before:** AI read word by word and forgot the beginning of long sentences.

**Attention:** AI sees the **entire text at once** and recognizes which words belong together.

“The **bank** by the **river** was cold.” → Attention understands: bench, not financial institution.

Every AI tool you use today is based on this invention.

## Attention Is All You Need

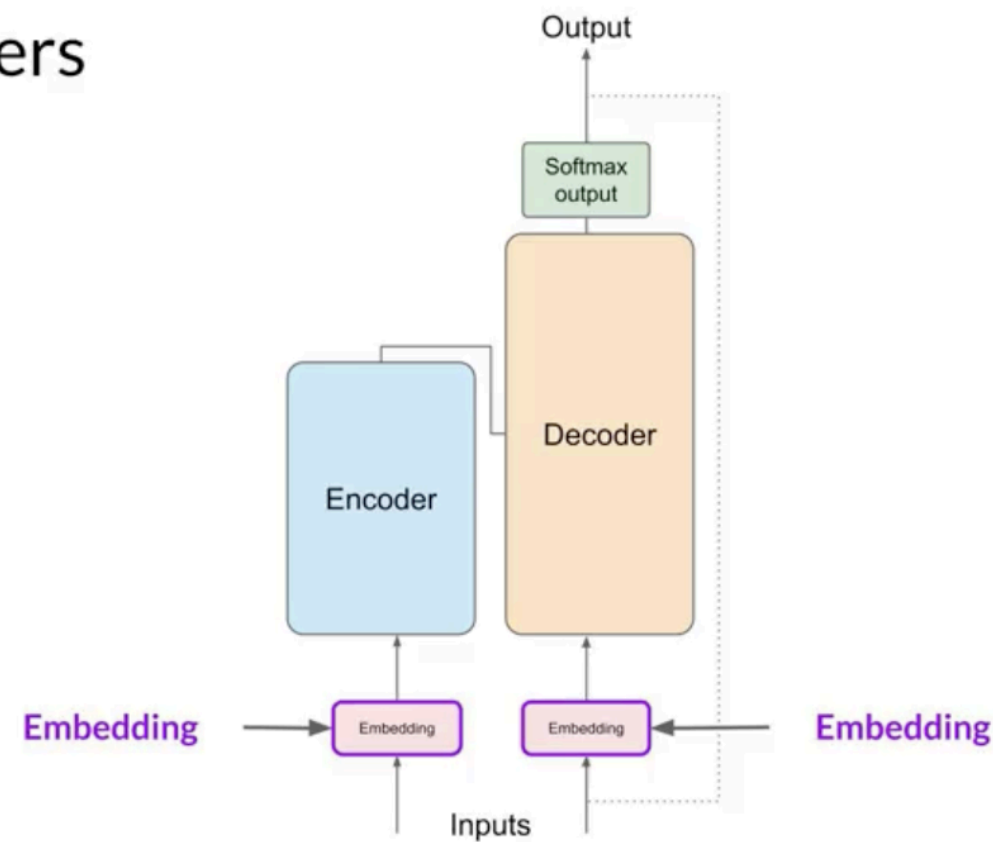
|                                                        |                                                                    |                                                           |                                                       |
|--------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------|
| Ashish Vaswani*<br>Google Brain<br>avaswani@google.com | Noam Shazeer*<br>Google Brain<br>noam@google.com                   | Niki Parmar*<br>Google Research<br>nikip@google.com       | Jakob Uszkoreit*<br>Google Research<br>usz@google.com |
| Llion Jones*<br>Google Research<br>llion@google.com    | Aidan N. Gomez* †<br>University of Toronto<br>aidan@cs.toronto.edu | Lukasz Kaiser*<br>Google Brain<br>lukaszkaizer@google.com |                                                       |
| Illia Polosukhin* ‡<br>illia.polosukhin@gmail.com      |                                                                    |                                                           |                                                       |

### Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

# The Transformer architecture

## Transformers



**Encoder** (left): Reads the input, understands the context

**Decoder** (right): Generates the output, token by token

## Who uses what?

| Task                       | Architecture      | Example                 |
|----------------------------|-------------------|-------------------------|
| Generate text              | Decoder only      | ChatGPT, Claude, Gemini |
| Translate (input → output) | Encoder + Decoder | Google Translate, DeepL |

# Attention: Focus on what matters

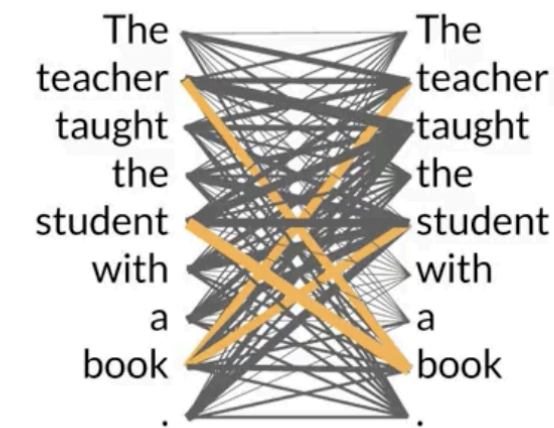
The **self-attention** mechanism allows AI to establish relationships:

“The technician repaired the **sensor** because **it** was defective.”

When it sees “it”, attention looks back to “sensor” and understands the reference.

Without attention, language would just be a pile of isolated words.

Self-attention



# The basic principle: next-token prediction

LLMs are extremely good **autocompletion systems**: the model calculates a probability for every possible next word and selects the most likely one.

What a beautiful day, the sun is

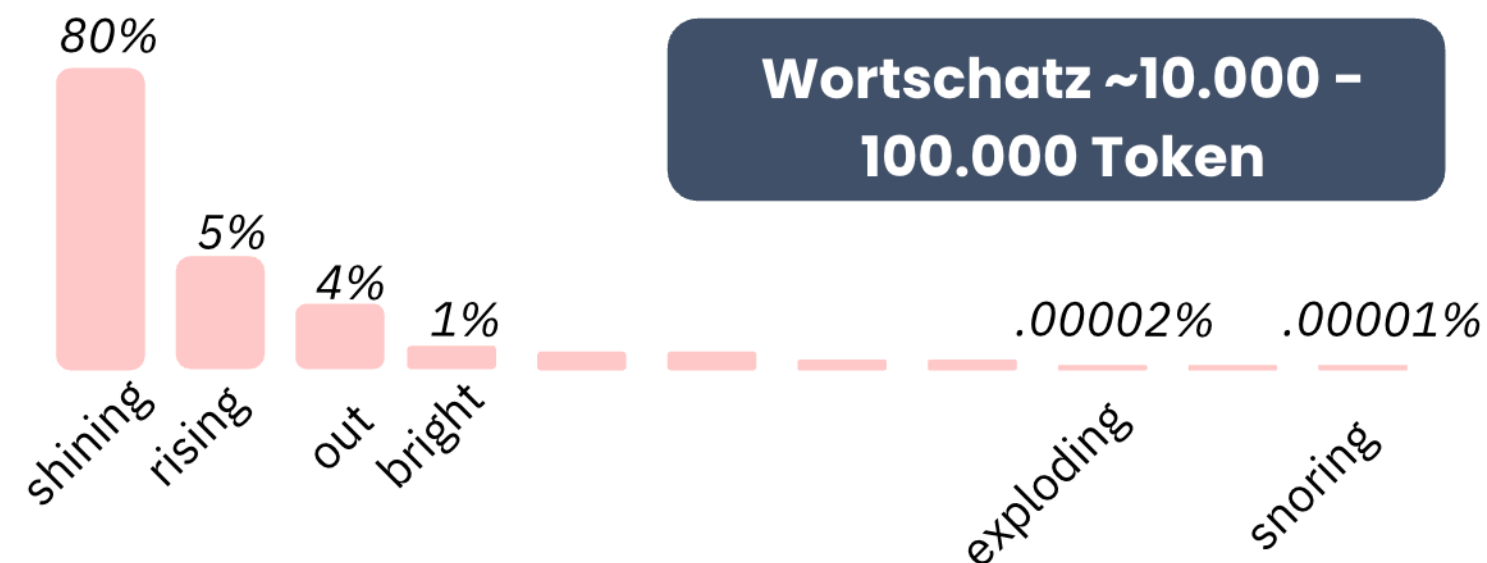
Verarbeite den  
aktuellen Text



Berechne  
Wahrscheinlichkeit



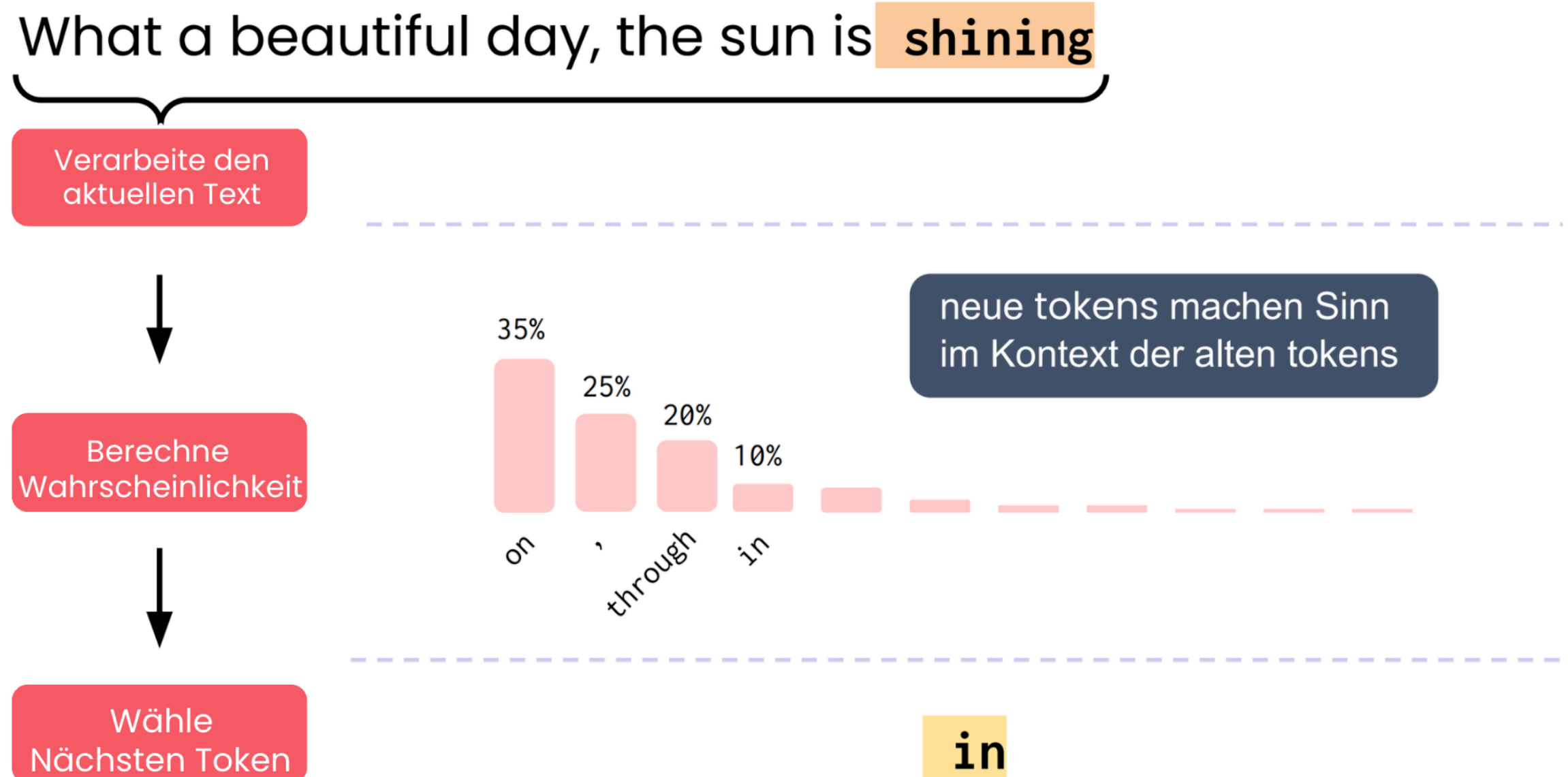
Wähle  
Nächsten Token



**shining**

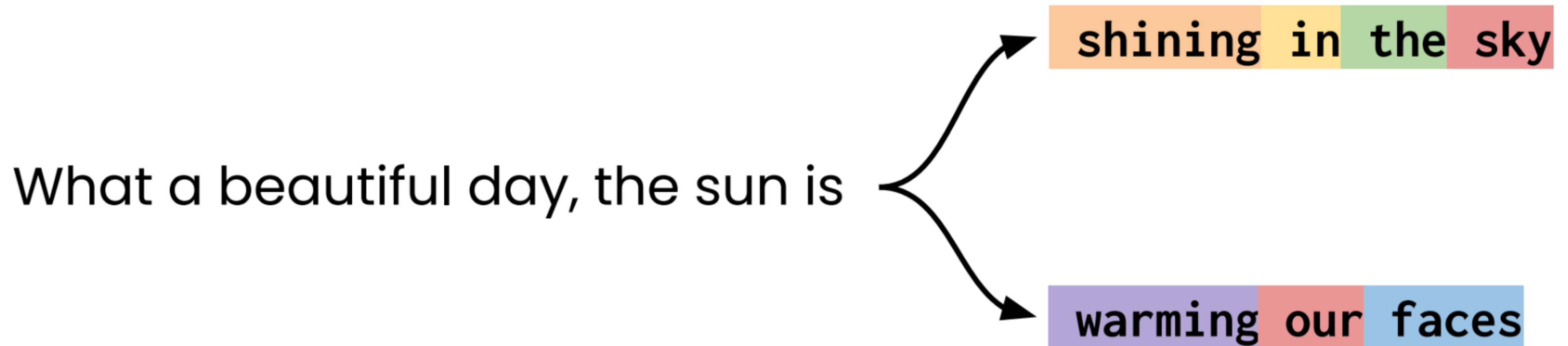
# Context determines the next token

Each new word influences the prediction. New tokens only make sense **in the context of the previous tokens**.



# Autoregressive generation

The same prompt can lead to completely different results:



- **Autoregressive** = “self-influencing”. Each token builds on the previous ones.
- That is why the same question does not always produce the same answer.



# The reasoning revolution (CoT)

Normal LLMs respond immediately. **Reasoning models** think first.

## How does it work?

Before answering, the model generates hundreds of **invisible intermediate tokens** in a `<think>` block. This is where the following happens:

-  **Break down:** Divide the problem into sub-steps
-  **Check:** Validate intermediate results
-  **Correct:** Detect errors and start over

# Reasoning: example

**Question:** “What is cheaper, 3 months at €49/month or 1 annual subscription for €499?”

| Standard LLM                          | Reasoning LLM                                                                                                                                                  |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prompt → Answer                       | Prompt → <think>...</think> → Answer                                                                                                                           |
| “The annual subscription is cheaper.” | <i>thinks: <math>3 \times 49 = 147</math> for 3 months, annualized 588€/year. <math>499 &lt; 588</math>. “The annual subscription saves you €89 per year.”</i> |

 **For practice**

Reasoning models take longer and cost more, but they are worth it for **math, analysis, and complex logic**. For simple questions, a standard LLM is enough.



# Tokens: The currency of AI

AI does not read letters, but **tokens**.

- 1 token  $\approx$  4 characters (in English)
- 1 token  $\approx$  0.75 words (in German)
- **Rule of thumb:** 1,000 tokens  $\approx$  750 words

"Donaudampfschiffahrt" → Donau | dampf | schiff | fahrt

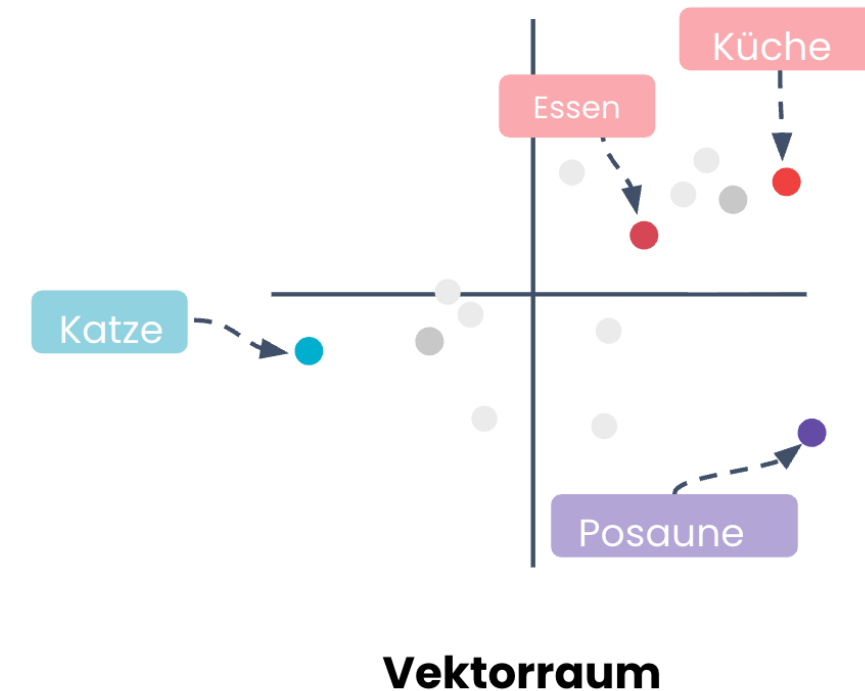
## Remember

Tokens are the atomic units of processing. Short, common words are often 1 token. Long, rare words are split up.

# Embeddings: Words as numeric vectors

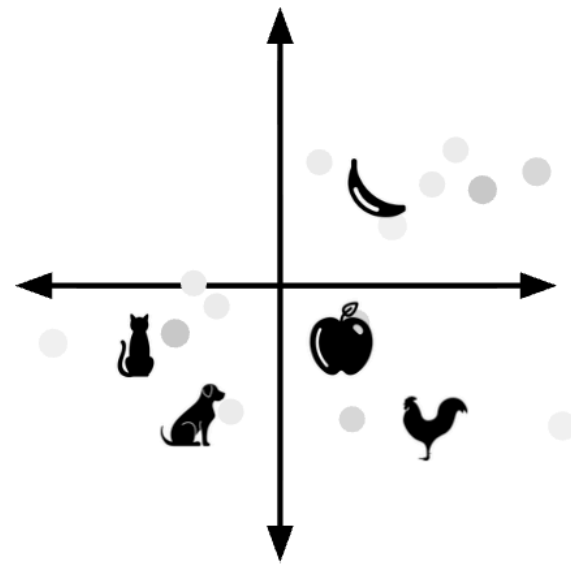
AI converts each word into a **numeric vector** and places it in a space.

- “Food” → vector **[3, 1]**
- “Kitchen” → vector **[5, 2]**
- **Similar meaning** = close together
- **Different meaning** = far apart

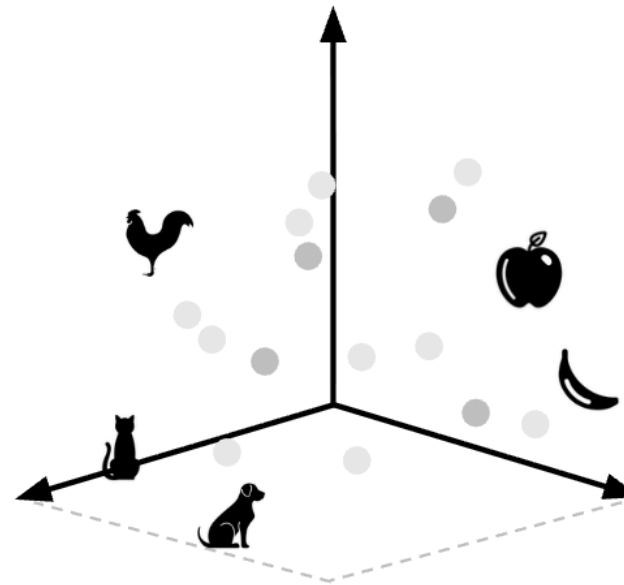


# More dimensions, more understanding

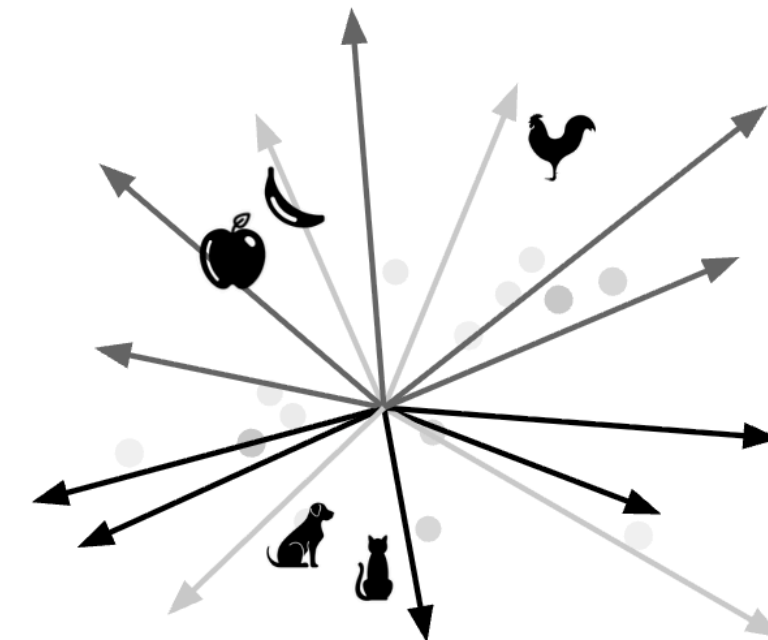
In reality: not 2, but **hundreds to thousands of dimensions**.



2 Dimensionen



3 Dimensionen



100 - 1.000+ Dimensionen

Mehr Dimensionen bieten zusätzlichen Raum für die Bildung von Clustern und die Erfassung nuancierter Beziehungen

Gleiche Prinzipien finden Anwendung. Nahe Vektoren weisen ähnliche Bedeutungen auf.

More dimensions = more space for **clusters** and **nuanced relationships**. The principle remains the same: nearby vectors = similar meaning.

# The context window: short-term memory

How much information can AI keep “in mind” at the same time?

| Model                           | Context           | Roughly equivalent to           |
|---------------------------------|-------------------|---------------------------------|
| GPT-4 (2023)                    | 128,000 tokens    | A short book                    |
| GPT-5.4 / Opus 4.6 / Gemini 3.1 | 1,000,000 tokens  | 3 complete Harry Potter volumes |
| Llama 4 Scout                   | 10,000,000 tokens | An entire corporate library     |

⚠ In 2 years: 1000x more context!

In 2023, one book fit. In 2026, **entire company archives** fit. That changes how we work with data.

# A first definition

- language becomes data
- data becomes representation
- representation becomes action
- action becomes a product, a prediction, or a decision

# Why language is hard for machines

## Humans do this easily

---

- resolve ambiguity
- use context
- understand sarcasm
- infer missing meaning
- adapt to domain and tone

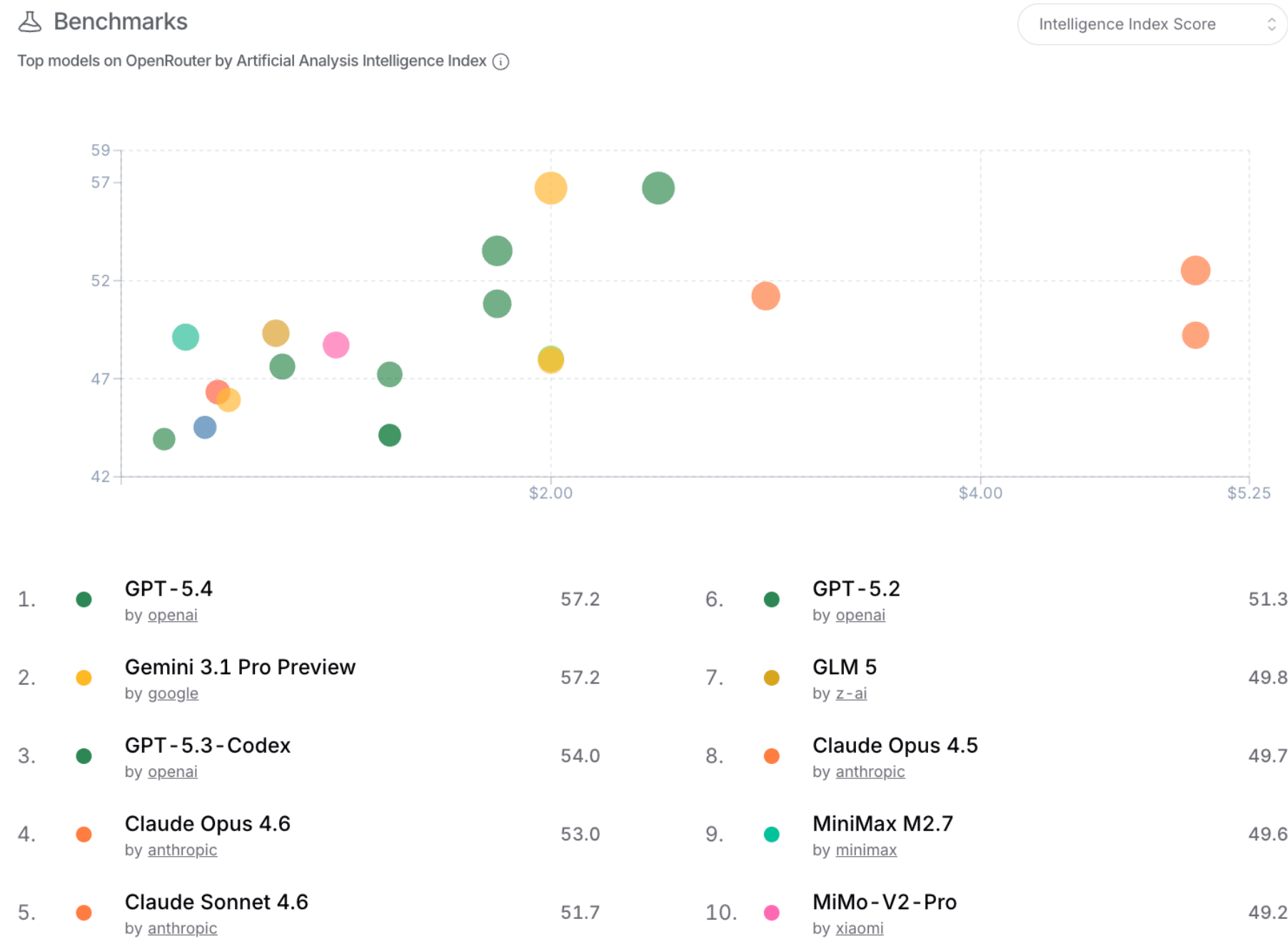
## Machines struggle because

---

- words have multiple meanings
- context changes interpretation
- language is messy and incomplete
- rules are rarely fully explicit
- meaning depends on world knowledge

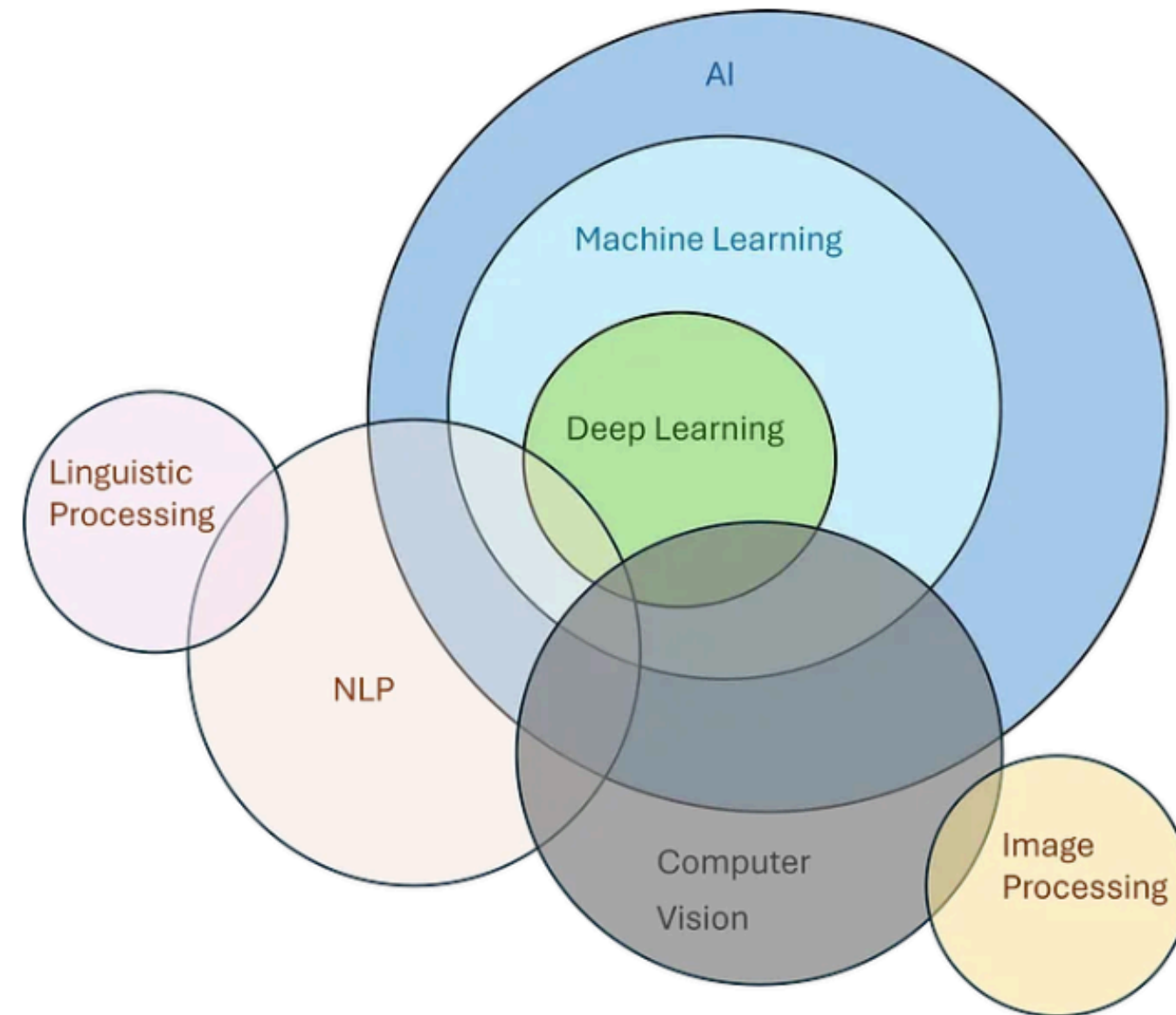


# Model landscape, March 2026



Source: OpenRouter / Artificial Analysis Intelligence Index, March 2026

# NLP inside the AI landscape



NLP is **not a subset** of Deep Learning. It spans from rule-based approaches (Linguistic Processing) through classical ML to modern deep learning (Transformers).



# Understanding the landscape

## AI $\supset$ ML $\supset$ Deep Learning

Concentric circles: each is a subset of the one above. Deep Learning uses neural networks with many layers.

## NLP overlaps all three

Rule-based NLP (regex, grammars) sits outside ML.  
Classical NLP (TF-IDF, Naive Bayes) uses ML.  
Modern NLP (Transformers, LLMs) uses Deep Learning.

## This course

We cover all three layers: classical preprocessing, ML-based classification, and transformer-powered NLP with Hugging Face.

# Classical NLP and modern NLP

| Dimension      | Classical NLP                                | Modern NLP                                   |
|----------------|----------------------------------------------|----------------------------------------------|
| Main logic     | rules, dictionaries, handcrafted features    | data-driven learning with neural networks    |
| Typical models | n-grams, TF-IDF, logistic regression, CRFs   | transformers, encoder models, decoder models |
| Strengths      | interpretable, lightweight, useful baselines | powerful, flexible, strong generalization    |
| Weaknesses     | brittle, limited context, manual effort      | compute cost, opacity, risk of hallucination |

# A tiny example of ambiguity

## Same word, different meaning

---

Sentence A: I sat by the bank and watched the river.  
Sentence B: I went to the bank to ask for a loan.

- same token: **bank**
- different context
- different meaning
- NLP must learn or infer which meaning is intended

# What counts as natural language?

## Text forms

---

- emails
- reviews
- social media posts
- contracts
- support tickets
- research articles

## Speech-related forms

---

- voice assistants
- meeting transcripts
- call-center conversations
- interviews
- podcasts

# Core NLP task families

## **Classification**

spam detection, topic  
tagging, sentiment labels

## **Extraction**

named entities, key  
phrases, facts

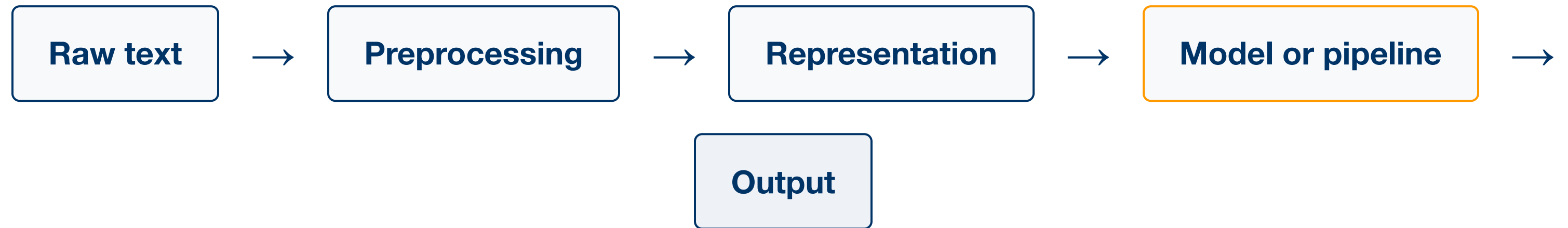
## **Retrieval**

search, ranking, semantic  
similarity, RAG

## **Generation**

summaries, answers,  
translations, new text

# The basic pipeline idea





# Tokenization

## Tokenization

Tokenization means splitting text into smaller units, such as **words, subwords, or sentences**, so a model can process it.

```
"NLP is fun!" → ["NLP", "is", "fun", "!" ]
```

- simple in principle
- not always simple in practice
- punctuation matters
- subword tokenization matters a lot for transformers

# Sentence vs. word vs. subword tokens

| Token type      | Example use                 | Why it matters                            |
|-----------------|-----------------------------|-------------------------------------------|
| Sentence tokens | summarization, segmentation | preserve larger meaning units             |
| Word tokens     | classical preprocessing     | easy to interpret                         |
| Subword tokens  | transformers                | handle rare words and shared stems better |



# Stopword removal

## Example

---

```
Original:  "The weather is really nice today."  
Filtered:  ["weather", "really", "nice", "today"]
```

- often useful in classical NLP
- not always useful in modern transformer pipelines
- choice depends on task and model

# Stopword removal, caution



## **Do not turn preprocessing into religion**

A step that helps one model can hurt another. Always think about the specific task.

# Lemmatization and stemming

## Lemmatization

---

- maps to a meaningful base form
- **running** → **run**
- **better** → **good**
- linguistically cleaner

## Stemming

---

- crude truncation of word forms
- faster, rougher, sometimes ugly
- can improve efficiency in simple pipelines

# Named Entity Recognition (NER)

## NER

Named Entity Recognition detects spans such as **people, organizations, locations, dates, currencies**, and more.

```
"Paul teaches in Bolzano in April 2026."
```

```
PERSON: Paul
```

```
LOCATION: Bolzano
```

```
DATE: April 2026
```

# Normalization and cleaning

- lowercase conversion
- punctuation handling
- number handling
- removing HTML, emojis, or noise when needed
- standardizing text before analysis

# Normalization and cleaning, caution

## Caution

But not every task wants aggressive cleaning. Sometimes the “mess” carries meaning.



# Core concepts as a workflow

| Step               | Question                      | Example                              |
|--------------------|-------------------------------|--------------------------------------|
| Tokenize           | How do we split text?         | words, sentences, subwords           |
| Clean              | What noise should be removed? | stopwords, punctuation, HTML         |
| Normalize          | How do we reduce variation?   | lowercase, lemma, standardized forms |
| Represent          | How does text become numbers? | TF-IDF, embeddings                   |
| Predict or extract | What is the task?             | sentiment, NER, classification       |

# A practical rule of thumb



## When in doubt

For classical models, preprocessing is often a **big part of performance**. For transformer pipelines, model choice and task framing often matter more.



**Final slide**

## **Introduction to NLP**

---

Thank you.

## Next time

Next time, we move from foundations to **classification and sentiment analysis**.

